192425192

G Vishnu Madhav

EXP22:

```
#include <stdio.h>
#include <string.h>

/* ---------- Simplified DES (8-bit block, 10-bit key) ---------- */
static const int P10[10] = {3,5,2,7,4,10,1,9,8,6};
static const int P8[8]   = {6,3,7,4,8,5,10,9};
static const int IPT[8]  = {2,6,3,1,4,8,5,7};
static const int IPI[8]  = {4,1,3,5,7,2,8,6};
static const int EP[8]   = {4,1,2,3,2,3,4,1};
static const int P4[4]   = {2,4,3,1};
static const int S0[4][4] = {{1,0,3,2},{3,2,1,0},{0,2,1,3},{3,1,3,2}};
static const int S1[4][4] = {{0,1,2,3},{2,0,1,3},{3,0,1,0},{2,1,0,3}};

static void perm(const int *t, int n, const int *in, int *out) {
    int i; for (i = 0; i < n; i++) out[i] = in[t[i] - 1];
}
static void rol(int *h, int n) {            /* rotate a 5-bit half left by n */
    int t[5], i; memcpy(t, h, sizeof t);
    for (i = 0; i < 5; i++) h[i] = t[(i + n) % 5];
}
static void sdes_keys(const int key[10], int k1[8], int k2[8]) {
    int p[10], i;
    perm(P10, 10, key, p);
    rol(p, 1); rol(p + 5, 1);
    int tmp[10]; memcpy(tmp, p, sizeof tmp); perm(P8, 8, tmp, k1);
    rol(p, 2); rol(p + 5, 2);
    perm(P8, 8, p, k2);
```

```
}
static void fk(const int *L, const int *R, const int *k, int *outL) {
    int e[8], i, r, c, s0, s1, p4in[4], p4out[4];
    perm(EP, 8, R, e);
    for (i = 0; i < 8; i++) e[i] ^= k[i];
    r = (e[0] << 1) | e[3]; c = (e[1] << 1) | e[2]; s0 = S0[r][c];
    r = (e[4] << 1) | e[7]; c = (e[5] << 1) | e[6]; s1 = S1[r][c];
    p4in[0] = s0 >> 1; p4in[1] = s0 & 1; p4in[2] = s1 >> 1; p4in[3] = s1 & 1;
    perm(P4, 4, p4in, p4out);
    for (i = 0; i < 4; i++) outL[i] = L[i] ^ p4out[i];
}
/* encrypt (dec=0) or decrypt (dec=1) one 8-bit block */
static void sdes(const int in[8], int out[8], const int key[10], int dec) {
    int k1[8], k2[8], ip[8], L[4], R[4], nl[4], pre[8], i;
    const int *ka, *kb;
    sdes_keys(key, k1, k2);
    ka = dec ? k2 : k1; kb = dec ? k1 : k2;
    perm(IPT, 8, in, ip);
    memcpy(L, ip, 16); memcpy(R, ip + 4, 16);
    fk(L, R, ka, nl);                   /* round 1 */
    memcpy(L, R, 16); memcpy(R, nl, 16);    /* switch */
    fk(L, R, kb, nl);                   /* round 2 */
    for (i = 0; i < 4; i++) { pre[i] = nl[i]; pre[4 + i] = R[i]; }
    perm(IPI, 8, pre, out);
}
static void print_bits(const char *label, const int *b, int n) {
    int i; printf("%s", label);
    for (i = 0; i < n; i++) { printf("%d", b[i]); if (i % 4 == 3 && i != n - 1) printf(" "); }
    printf("\n");
}
```

```
int main() {
    char ks[40], ivs[40], pts[200];
    int key[10], iv[8], pt[200], ct[200], dt[200], prev[8], x[8], n, i, j, nb;

    printf("Enter 10-bit key              : ");
    scanf("%39s", ks);
    printf("Enter 8-bit initialization vector: ");
    scanf("%39s", ivs);
    printf("Enter binary plaintext        : ");
    scanf("%199s", pts);

    for (i = 0; i < 10; i++) key[i] = ks[i] - '0';
    for (i = 0; i < 8; i++)  iv[i]  = ivs[i] - '0';
    n = strlen(pts); nb = n / 8;
    for (i = 0; i < n; i++) pt[i] = pts[i] - '0';

    /* CBC encryption:  C_i = E(K, P_i XOR C_{i-1}),  C_0 = IV */
    memcpy(prev, iv, sizeof prev);
    for (i = 0; i < nb; i++) {
        for (j = 0; j < 8; j++) x[j] = pt[8 * i + j] ^ prev[j];
        sdes(x, ct + 8 * i, key, 0);
        memcpy(prev, ct + 8 * i, sizeof prev);
    }
    printf("\n");
    print_bits("Plaintext  : ", pt, n);
    print_bits("Ciphertext : ", ct, n);

    /* CBC decryption:  P_i = D(K, C_i) XOR C_{i-1} */
    memcpy(prev, iv, sizeof prev);
```

```
    for (i = 0; i < nb; i++) {
        sdes(ct + 8 * i, x, key, 1);
        for (j = 0; j < 8; j++) dt[8 * i + j] = x[j] ^ prev[j];
        memcpy(prev, ct + 8 * i, sizeof prev);
    }
    print_bits("Decrypted  : ", dt, n);
    return 0;
}
```

OUTPUT:

```
Enter 10-bit key              : 0111111101
Enter 8-bit initialization vector: 10101010
Enter binary plaintext        : 0000000100100011

Plaintext  : 0000 0001 0010 0011
Ciphertext : 1111 0100 0000 1011
Decrypted  : 0000 0001 0010 0011
```